G05IART0.02 - Inteligência Artificial
Trabalho 2 - Aprendizado de Máquina e Computação Natural
Entrega: 03/12/2026 - Valor: 20,0 pontos

1. Apresentação e objetivos

Neste trabalho, cada grupo desenvolverá duas investigações computacionais. Na **Parte 1**, deverá escolher uma **base de dados real no UCI Machine Learning Repository ou no Kaggle**, preparar os dados e comparar algoritmos de aprendizado supervisionado, incluindo métodos *ensemble*. Na **Parte 2**, deverá escolher **um problema da lista oficial de 50 problemas de otimização** disponibilizada para a turma e abordá-lo com **Algoritmo Genético (AG), um método de Inteligência de Enxame e um Sistema Imunológico Artificial**. Os três métodos de otimização devem resolver **o mesmo problema, as mesmas instâncias e a mesma função objetivo**.

O propósito é demonstrar como escolhas de representação, preparo de dados, operadores, hiperparâmetros e procedimentos experimentais afetam a qualidade dos resultados. As partes podem tratar de domínios diferentes; **não é necessário que o dataset da Parte 1 esteja relacionado ao problema de otimização da Parte 2**.

2. Organização do trabalho

1. **Parte 1 - Aprendizado supervisionado e ensembles (8,0 pontos):** Árvore de Decisão, KNN, SVM, Random Forest e AdaBoost, aplicados à mesma base.
2. **Parte 2 - Computação Natural (8,0 pontos):** formulação do problema e implementação própria de AG, um método de enxame e um método imune.
3. **Parte 3 - Comparação, documentação e entrega (4,0 pontos):** protocolo experimental, análise dos resultados, relatório e reprodutibilidade.

3. Parte 1 - Base UCI/Kaggle e aprendizado supervisionado

3.1. Escolha da base e justificativa

O grupo deverá selecionar uma base do UCI Machine Learning Repository ou do Kaggle. Registrar **nome, endereço de origem, autoria quando disponível, licença/condições de uso, descrição do domínio, número de instâncias, atributos, variável-alvo e tipo de tarefa**. É recomendada uma **tarefa de classificação**, pois permite comparar os cinco classificadores exigidos. Uma base de regressão será aceita somente se **aprovada previamente pelo professor** e se os cinco métodos forem adaptados de maneira consistente às suas versões de regressão; nesse caso, o grupo deverá justificar as escolhas e usar métricas de regressão.

A base deverá ser suficientemente informativa para a comparação. Não serão aceitas, sem autorização, bases meramente demonstrativas ou sintéticas triviais. Antes da implementação, o grupo deve demonstrar que conhece o significado da variável-alvo, as características dos atributos e eventuais limitações da coleta. **Não escolher a base exclusivamente pela facilidade de obter alta acurácia.** A seleção deverá ser registrada no README e apresentada ao professor, com breve justificativa, antes da experimentação definitiva.

3.2. Análise exploratória de dados (EDA)

Apresentar, no mínimo: quantidade de amostras e atributos; tipos de variáveis; distribuição do alvo (e desbalanceamento de classes, quando houver); valores ausentes; duplicidades; inspeção de atributos

numéricos e categóricos; gráficos/tabelas com interpretações. Explicar as decisões sobre remoção de registros, imputação, codificação de categorias, tratamento de valores extremos e seleção de atributos, quando utilizados.

3.3. Preparo dos dados e validação

- Separar **treino e teste** antes de ajustar transformações que aprendem dos dados; reservar o conjunto de teste para avaliação final.
- Usar divisão estratificada em classificação, quando viável. Empregar validação cruzada no treino (recomenda-se **5 folds**); justificar adaptações a bases pequenas, temporais ou agrupadas.
- Aplicar imputação, codificação e escalonamento **dentro de** `Pipeline/ColumnTransformer` ou procedimento equivalente, ajustados apenas nos folds de treino, evitando *data leakage*.
- Usar **o mesmo particionamento e protocolo de avaliação** para todos os modelos. Documentar sementes, hiperparâmetros, estratégia de ajuste e eventuais desequilíbrios da base.

3.4. Modelos obrigatórios

1. **Árvore de Decisão** (`DecisionTreeClassifier`): avaliar, por exemplo, profundidade e critério de divisão; interpretar regras ou visualização de uma árvore.
2. **KNN** (`KNeighborsClassifier`): investigar ao menos três valores de k; discutir influência do escalonamento.
3. **SVM** (`SVC`): comparar ao menos duas configurações justificadas (por exemplo, kernels ou valores de C); discutir escalonamento.
4. **Random Forest** (`RandomForestClassifier`): investigar número de árvores e ao menos um parâmetro de complexidade; explicar agregação e aleatoriedade em *bagging*.
5. **AdaBoost** (`AdaBoostClassifier`): investigar número de estimadores e/ou taxa de aprendizado; explicar o treinamento sequencial e a ponderação dos exemplos.

O uso do `scikit-learn` **é permitido na Parte 1**. Não se exige implementação do zero desses cinco algoritmos. Ajustes de hiperparâmetros deverão ser feitos exclusivamente com dados de treino, nunca com o teste final. Métodos adicionais, como XGBoost ou Gradient Boosting, são opcionais e **não substituem** os cinco obrigatórios.

3.5. Métricas e discussão

Para classificação, apresentar **acurácia, precisão macro, revocação macro e F1 macro**, além de **matriz de confusão** para ao menos os modelos de maior interesse. Em classificação com classes desbalanceadas, discutir por que acurácia isoladamente pode induzir conclusões incorretas. ROC-AUC pode ser incluída, quando aplicável. Para regressão previamente autorizada, utilizar **MAE, RMSE e** R^2. Reportar resultados da validação cruzada e resultados finais no conjunto de teste, identificando claramente cada um.

A discussão deve responder: (a) qual modelo apresentou melhor desempenho e segundo qual métrica? (b) Random Forest e AdaBoost superaram a árvore individual? (c) como as decisões de pré-processamento influenciaram KNN e SVM? (d) quais limitações dos dados e da metodologia permanecem? Não afirmar que um *ensemble* é necessariamente superior.

4. Parte 2 - Um problema para AG, Enxame e Imunes

4.1. Escolha entre os 50 problemas oficiais

Cada grupo escolherá **um único problema da lista oficial de 50 problemas de otimização da disciplina**, que constitui o conjunto autorizado de temas desta parte. Identificar o **número e o nome exatamente como constam na lista original**, além das condições e instâncias empregadas. Não substituir o problema por outro sem autorização. Diferentes grupos podem receber problemas distintos conforme orientação do professor.

A lista completa, com a numeração original e adaptações propostas para os três métodos, encontra-se no **Apêndice A**. O protocolo comum de implementação está no **Apêndice B**.

4.2. Formulação comum obrigatória

Antes de programar, formalizar o problema: (i) variáveis de decisão e domínio; (ii) solução candidata e representação; (iii) função objetivo fx e se é minimização ou maximização; (iv) restrições e como será garantida ou recuperada a viabilidade; (v) definição de instâncias e critério de parada. Descrever uma forma verificável de calcular o custo/fitness de uma solução.

Os três métodos deverão usar a mesma definição de solução válida, as mesmas instâncias e a mesma função objetivo original. Representações internas e operadores poderão variar conforme o método, desde que haja uma conversão inequívoca para soluções comparáveis. Penalizações devem ser uniformizadas ou reportadas separadamente do custo original; informar sempre se uma solução é viável.

4.3. Algoritmo Genético (AG) - implementação própria

Implementar do zero: inicialização de população, codificação, avaliação de fitness, seleção, cruzamento, mutação, estratégia de substituição e critério de parada. Justificar probabilidades, tamanho da população, elitismo (se houver) e operadores adequados ao problema. A saída deve apresentar ao menos a melhor solução, seu valor objetivo original, viabilidade e histórico de convergência.

4.4. Inteligência de Enxame - implementação própria

Escolher e implementar **PSO** (Otimização por Enxame de Partículas) **ou ACO** (Otimização por Colônia de Formigas), conforme a natureza do problema. Para PSO: representar posição, velocidade, limites, atualização individual e coletiva, e adaptação necessária em problemas discretos. Para ACO: definir construção da solução, trilhas de feromônio, informação heurística, atualização e evaporação. **Não basta selecionar um método: é preciso justificar sua adequação e viabilidade para o problema escolhido.** Para variantes discretas de PSO, documentar explicitamente a transformação de posição em decisão válida; para ACO, documentar o espaço de construção, as probabilidades de transição e a atualização de feromônio. Apenas arredondar valores sem analisar restrições e viabilidade não caracteriza uma adaptação suficiente.

4.5. Sistema Imunológico Artificial - implementação própria

Implementar **CLONALG** (seleção clonal) ou propor outro método imune **mediante aprovação prévia**, quando a adequação exigir. Para CLONALG: definir população de anticorpos, afinidade, seleção, clonagem, hipermutação, atualização da população e parada. A afinidade poderá transformar monotonicamente o custo original, desde que os resultados finais sejam reportados na **mesma escala da função objetivo** dos outros métodos. Se o problema tiver restrições, explicitar como a clonagem/hipermutação preserva ou recupera soluções válidas.

4.6. Restrições de implementação

Os métodos AG, Enxame e Imunes deverão ser **implementados pelo grupo**. São permitidos Python, NumPy, SciPy, pandas e bibliotecas para visualização, testes e suporte matemático. **Não são permitidas bibliotecas que forneçam implementações prontas dos meta-heurísticos exigidos** (por exemplo, invocar um otimizador genético, PSO, ACO ou CLONALG completo). Bibliotecas de avaliação objetiva ou manipulação de grafos poderão ser utilizadas apenas quando não substituírem o núcleo dos algoritmos, com sua função claramente documentada.

5. Comparação experimental obrigatória

Na **Parte 1**, comparar os cinco modelos com os mesmos dados, partições e métricas. É obrigatório informar a estratégia de seleção de hiperparâmetros; o conjunto de teste não pode ser empregado para selecionar modelos, atributos ou parâmetros. Na **Parte 2**, comparar AG, Enxame e Imunes por **qualidade da solução, viabilidade, robustez e esforço computacional**.

Para os meta-heurísticos, realizar **no mínimo 10 execuções independentes por algoritmo e por instância avaliada**, utilizando sementes diferentes e registradas. Relatar **melhor valor, média e desvio-padrão (ou mediana e intervalo interquartil), taxa de soluções viáveis, tempo de execução e número de avaliações da função objetivo**. Apresentar **curvas de convergência** em função de avaliações da função objetivo; gráficos por iteração podem ser acrescentados.

Para maior justiça experimental, os métodos devem ter um **orçamento comparável de avaliações da função objetivo**, definido e informado previamente pelo grupo. O custo de avaliação deve incluir avaliações feitas durante inicialização, produção de candidatos, clonagem, mutação, movimentação de partículas e construção de soluções, conforme o método. Não utilizar o número de iterações como única base de comparação. Relatar configurações e hardware utilizado; reconhecer quando a comparação não puder ser perfeitamente equilibrada. Definir as instâncias e orçamentos **antes** de analisar os resultados. Incluir uma referência simples (solução aleatória viável, solução construtiva ou resultado conhecido) para contextualizar a qualidade das soluções; essa referência não substitui os três métodos exigidos.

6. Entregáveis e organização do repositório

O grupo deverá entregar **um repositório Git público** (GitHub, GitLab ou equivalente), com código e instruções reproduzíveis, e **um relatório em PDF**. No SIGAA, enviar **o endereço do repositório e o relatório PDF** até **03/12/2026**. Criar uma **tag** `v1.0-submissao` correspondente à versão entregue. Nenhuma parte do código deve depender de caminhos absolutos da máquina do estudante.

```
trabalho-ia-2026/
  README.md
  requirements.txt
  Makefile                 # opcional; ou run.sh
  data/
    README.md             # origem, licença e download
    raw/                  # somente se distribuição permitida
    processed/
  notebooks/
    eda.ipynb
  src/
    supervised/
      preprocess.py
      evaluate.py
      train_models.py
    optimization/
      problem.py          # funcao objetivo e restricoes
      genetic.py
```

```
      swarm.py
      immune.py
      experiment.py
  results/
    tables/
    figures/
  tests/
  reports/
    relatorio.pdf
```

Listing 1. Estrutura de repositório sugerida (adaptável).

O `README.md` deve informar: membros do grupo; identificação e origem da base e do problema de otimização; requisitos; instalação; procedimentos de execução separados para cada parte; sementes e parâmetros; como recriar tabelas e figuras. Não publicar bases que violem licenças, dados pessoais ou arquivos excessivamente grandes: nesses casos, incluir instruções ou script legítimo de obtenção dos dados. Evitar armazenar credenciais no repositório.

7. Relatório técnico

Produzir um relatório de **6 a 10 páginas de conteúdo principal** (referências e apêndices não contam), contendo:

1. Introdução e identificação do grupo.
2. Parte 1: origem da base, justificativa, caracterização da tarefa, EDA, pré-processamento, protocolo de validação, modelos, tabelas de métricas e análise crítica.
3. Parte 2: identificação do problema na lista oficial, formulação matemática, função objetivo, restrições, representação, operadores dos três algoritmos e parâmetros.
4. Experimentos: ambiente, sementes, orçamentos, tabelas comparativas, gráficos de convergência, discussão de viabilidade e variabilidade.
5. Conclusão: quais escolhas funcionaram melhor, limitações observadas e possíveis melhorias.
6. Referências e instruções de reprodução (ou indicação da seção correspondente do README).

Gráficos devem conter títulos/legendas, nomes dos eixos e unidades. Não utilizar somente capturas de tela de execução como apresentação de resultados.

8. Seleção, viabilidade e escopo dos problemas

A lista do Apêndice A contém **50 temas disponíveis**, não 50 implementações prontas ou equivalentes em dificuldade. Cada grupo deverá registrar uma **proposta técnica de uma página ou seção no README** contendo: número do problema, instâncias planejadas, variáveis, função objetivo, restrições, representação e operadores preliminares de AG, PSO/ACO e CLONALG. A proposta deverá ser apresentada ao professor em atividade de acompanhamento; escolhas inviáveis deverão ser ajustadas **sem trocar o objetivo central** ou mediante autorização.

Escopo didático: serão aceitas instâncias reduzidas, modelos matemáticos simplificados e simuladores reprodutíveis, desde que declarados. O grupo não precisa resolver uma versão industrial completa do tema, mas deve preservar um problema de otimização genuíno, decisões não triviais, restrições verificáveis e uma função objetivo mensurável. Trabalhos que consistam apenas em gerar números aleatórios sem vínculo com o domínio escolhido não satisfazem o enunciado.

Classificação orientativa de esforço: os temas **12, 16, 18, 21, 26, 27, 29, 35, 37, 49 e 50** merecem **validação de escopo obrigatória** antes da implementação por envolverem treinamento interno de modelos, simulação especializada ou codificação estrutural mais complexa. Os demais também deverão justificar as escolhas de representação. Essa classificação não proíbe nenhum dos 50 temas e não implica bonificação automática.

Critérios mínimos de validação: testar separadamente o avaliador e as restrições, apresentar pelo menos uma solução sabidamente viável, uma solução inválida e uma pequena instância controlada. Se um método exigir decodificação ou reparo, demonstrar que a solução decodificada corresponde ao objetivo original. Alterações metodológicas devem aparecer no relatório e não podem favorecer um algoritmo mudando o problema que ele resolve.

9. Cronograma de desenvolvimento

Data	Marco / orientação
14/10	Publicação do trabalho; início da escolha da base e do problema de otimização.
27/10	Meta recomendada: dataset identificado, EDA inicial e proposta de escopo do problema.
28/10	Atividade em classe: AG, representação, fitness e operadores.
04/11	Atividade em classe: método de enxame e adaptação ao problema.
11/11	Atividade em classe: método imune e planejamento da comparação.
17/11	Aula teórica de Ensemble Learning.
18/11	Atividade em classe: Random Forest, AdaBoost e avaliação da Parte 1.
24/11	Atividade em classe: integração, resultados e revisão experimental.
01/12	Prova II (conforme cronograma da disciplina).
03/12	**Entrega final no SIGAA: repositório e relatório.**

As datas intermediárias são **marcos orientativos de desenvolvimento**, não entregas avaliativas adicionais, salvo comunicação específica do professor. A aprovação de escopo poderá ocorrer durante o acompanhamento em classe, sem pontuação separada.

10. Rubrica de avaliação - 20,0 pontos

Critério	Pontos
Escolha da base, justificativa e análise exploratória	2,0
Pré-processamento, prevenção de vazamento e validação	2,0
Árvore de Decisão, KNN e SVM: aplicação e comparação	2,0
Random Forest e AdaBoost: aplicação e análise de ensembles	2,0
Formulação do problema de otimização: função objetivo e restrições	2,0
Implementação e parametrização do AG	2,0
Implementação e parametrização do método de Enxame	2,0
Implementação e parametrização do método Imune	2,0
Experimentos e comparação entre os três métodos de otimização	2,0
Relatório, organização e reprodutibilidade do repositório	2,0
Total	**20,0**

A pontuação depende da correção metodológica e da explicação do trabalho, **não de alcançar a maior acurácia ou o menor custo absoluto**. Resultados aparentemente excelentes sem protocolo válido, ou implementações que não executem, poderão perder pontos nos respectivos critérios.

11. Orientações finais e checklist

- □ A base foi escolhida no UCI ou Kaggle e sua fonte/uso estão documentados.
- □ Os cinco modelos supervisionados foram avaliados sob protocolo comum, sem vazamento de dados.
- □ O problema da Parte 2 está identificado **pelo número e nome da lista oficial dos 50 problemas**.
- □ AG, Enxame e Imunes resolvem o **mesmo problema e as mesmas instâncias**, com uma função objetivo comum.
- □ Os três meta-heurísticos foram implementados pelo grupo, não invocados de bibliotecas prontas.
- □ Existem ao menos 10 execuções independentes por método/instância, com resultados e sementes registrados.
- □ Tabelas, curvas, comparação e limitações aparecem no relatório.
- □ O repositório tem instruções de instalação/execução e a tag `v1.0-submissao`.
- □ O link do repositório e o PDF foram enviados no SIGAA até **03/12/2026**.

Observação: o código apresentado deve ser compreendido pelos integrantes do grupo. Decisões metodológicas e experimentais podem ser solicitadas e discutidas nas atividades em classe.

Apêndice A - 50 problemas e adaptações para AG, Enxame e Sistemas Imunológicos

Natureza das propostas. Os nomes e escopos dos 50 problemas abaixo foram preservados da lista original *50 ideias de problemas para Algoritmos Genéticos*. As formulações, representações e adaptações para Enxame e CLONALG apresentadas neste apêndice são **propostas didáticas para a edição de 2026**, não descrições presentes na lista original e não implementações já validadas.

Padrão comum. Cada grupo constrói um **avaliador único** que recebe uma solução decodificada e devolve: custo/benefício original, viabilidade e violações. Os três algoritmos usam as mesmas instâncias, entradas, restrições e avaliação; cada um pode ter operadores e códigos internos próprios. Soluções inválidas devem ser reparadas ou ter suas violações explicitamente penalizadas de maneira documentada, sem esconder a métrica original. AG usa seleção, recombinação e mutação; Enxame usa **ACO** para construção probabilística de sequências/percursos ou **PSO adaptado** (binário, discreto ou misto); Imunes usa **CLONALG**, com clonagem, seleção e hipermutação próprias ao domínio.

Limites de escopo. A lista contém casos de dificuldade desigual. Em problemas envolvendo simulação de engenharia, saúde, finanças, agentes ou modelos de aprendizado, exigir **simuladores, dados públicos ou instâncias sintéticas pequenas**, critérios verificáveis e nenhum controle físico ou decisão clínica real. As propostas 12, 16, 18, 21, 26, 27, 29, 35, 37, 49 e 50 exigem particular cuidado com custo computacional, definição da representação ou validade do simulador. Para o problema 16, os métodos exteriores otimizam um *AG interno fixo* em uma instância independente; o AG exterior continua sendo um algoritmo distinto, evitando circularidade metodológica. Para qualquer problema, o grupo deve validar pelo menos uma solução viável e um avaliador independente antes de comparar os métodos.

1. Caixeiro Viajante (TSP) para 10–100 cidades com restrições (janelas de tempo, cidades obrigatórias).

Objetivo: minimizar distância/atrasos.

Representação comum: permutação de cidades; horários como restrições.

Enxame: ACO: probabilidades de transição entre cidades.

CLONALG: troca/inversão de segmentos; reparo de janelas. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

2. Problema da Mochila (0/1 ou fracionária) com múltiplos tipos de itens e limite de peso/volume.

Objetivo: maximizar valor com peso/volume viáveis.

Representação comum: vetor binário ou quantidades discretizadas.

Enxame: PSO binário/discreto com arredondamento e reparo.

CLONALG: inverter bits ou alterar quantidades; reparar capacidade. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

3. Alocação de salas de aula distribuindo turmas em salas e horários minimizando choques.

Objetivo: minimizar choques e salas inadequadas.

Representação comum: vetor de sala e horário por turma.

Enxame: PSO discreto com decodificador de alocação.

CLONALG: realocar duas turmas; reparar conflitos. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

4. Escalonamento de funcionários (plantões) respeitando carga horária e folgas.

Objetivo: minimizar violações de carga e descobertura.

Representação comum: vetor de turno atribuído a cada funcionário.

Enxame: PSO discreto com reparo de cobertura.

CLONALG: troca de plantões e redistribuição. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

5. Roteamento de veículos (VRP) para entrega em vários pontos com limite de capacidade.

Objetivo: minimizar distância e veículos.

Representação comum: permutação de clientes e divisão em veículos.

Enxame: ACO: construção de rotas viáveis por veículo.

CLONALG: swap/realocação de cliente; reparar capacidade. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

6. Organização de cronograma de provas minimizando conflitos entre alunos.

Objetivo: minimizar conflitos e penalizações.

Representação comum: permutação de provas em horários/salas.

Enxame: ACO para ordem de provas com decodificação.

CLONALG: trocar provas de horário e corrigir conflitos. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

7. Problema de coloração de grafos minimizando o número de cores sem conflitos.

Objetivo: minimizar cores, exigindo coloração válida.

Representação comum: cor atribuída a cada vértice.

Enxame: PSO discreto para índices de cores.

CLONALG: recolorir vértice e reparar arestas conflitantes. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

8. Ordenação de tarefas em linha de produção para minimizar o tempo total (makespan).

Objetivo: minimizar makespan.

Representação comum: permutação de tarefas e máquinas designadas.

Enxame: ACO para sequência com decodificador.

CLONALG: inversão de tarefas; mudança de máquina. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

9. Formação de times de projeto equilibrando habilidades, carga e preferências.

Objetivo: minimizar desequilíbrio e preferências violadas.

Representação comum: identificador de equipe para cada integrante.

Enxame: PSO discreto com limites de tamanho.

CLONALG: mover/trocar integrantes entre equipes. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

10. Alocação de disciplinas para professores respeitando preferências e restrições de horário.

Objetivo: minimizar violações e insatisfação.

Representação comum: atribuição professor-disciplina-horário.

Enxame: PSO discreto com reparação de conflitos.

CLONALG: trocar disciplinas ou horários. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

11. Otimização de hiperparâmetros de um SVM (C, gamma, kernel) via AG.

Objetivo: maximizar F1-CV ou minimizar erro de validação.

Representação comum: vetor real log(C), log(gamma) e código de kernel.

Enxame: PSO misto com discretização do kernel.

CLONALG: perturbar parâmetros e trocar kernel. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

12. Escolha de arquitetura de rede neural (número de camadas, neurônios, dropout).

Objetivo: minimizar erro de validação + custo.

Representação comum: inteiros para camadas, neurônios; reais para dropout.

Enxame: PSO misto arredondando estrutura.

CLONALG: alterar camada, largura ou dropout. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

13. Seleção de features em um dataset usando uma string binária para indicar atributos.

Objetivo: maximizar escore CV e penalizar quantidade.

Representação comum: máscara binária de atributos.

Enxame: PSO binário por probabilidade sigmoide.

CLONALG: inverter inclusão de atributos. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

14. Sintonização de controladores PID (Kp, Ki, Kd) para sistemas de controle.

Objetivo: minimizar erro de controle simulado.

Representação comum: vetor real Kp, Ki, Kd em limites seguros.

Enxame: PSO contínuo com limites.

CLONALG: perturbação gaussiana por componente. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

15. Evolução de estratégias de busca (parâmetros de A*, peso da heurística, profundidade).

Objetivo: minimizar custo/tempo de busca em instâncias fixas.

Representação comum: vetor de parâmetros de heurística/limites.

Enxame: PSO contínuo ou misto com decodificação.

CLONALG: alterar parâmetros dentro do intervalo. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

16. Otimização dos parâmetros do próprio algoritmo genético (tamanho de população, taxas).

Objetivo: minimizar custo médio obtido pelo AG interno.

Representação comum: vetor de hiperparâmetros de um AG interno fixo.

Enxame: PSO misto com decodificador.

CLONALG: perturbar taxas e tamanho de população. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

17. Projeto de heurísticas de escalonamento do tipo regras IF–THEN.

Objetivo: minimizar custo do escalonamento.

Representação comum: pesos e limiares de um conjunto fixo de regras.

Enxame: PSO misto, regras codificadas por parâmetros.

CLONALG: alterar limiares e ativar/desativar regras. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

18. AutoML simples: escolher pipeline (normalização, PCA, modelo) para classificação.

Objetivo: maximizar validação e penalizar custo.

Representação comum: categorias do pipeline e hiperparâmetros.

Enxame: PSO misto com decodificação de escolhas.

CLONALG: trocar etapas/modelos e perturbar parâmetros. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

19. Evolução de estratégias para jogo da velha (política de jogadas).

Objetivo: maximizar vitórias contra adversários fixos.

Representação comum: tabela de preferência de ações por estado.

Enxame: PSO discreto sobre ações codificadas.

CLONALG: trocar ação em um estado. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

20. Evolução de pesos de uma heurística de xadrez simplificado (material + mobilidade).

Objetivo: maximizar desempenho em partidas fixas.

Representação comum: pesos reais de componentes da função de avaliação.

Enxame: PSO contínuo limitado.

CLONALG: perturbar pesos e renormalizar. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

21. Evolução de bots para jogo de corrida (aceleração, freio e curva) com rede simples.

Objetivo: maximizar progresso e minimizar colisões.

Representação comum: pesos/controladores discretos do bot.

Enxame: PSO contínuo ou misto.

CLONALG: perturbar pesos/limiares. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

22. Configuração de árvore de decisão para um jogo simples (adivinhação de número).

Objetivo: minimizar tentativas e complexidade.

Representação comum: estrutura fixa de árvore com testes/limiares.

Enxame: PSO misto com decodificação de nós.

CLONALG: alterar teste, limiar ou folha. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

23. Evolução de regras para um agente em labirinto (sensores locais e ações).

Objetivo: maximizar êxito e minimizar passos.

Representação comum: tabela de ações por observação local.

Enxame: PSO discreto com decodificador de ações.

CLONALG: alterar regra/ação em uma observação. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

24. Design automático de níveis de jogo (posição de obstáculos e inimigos).

Objetivo: maximizar dificuldade com níveis solucionáveis.

Representação comum: grade de tipos de células/obstáculos.

Enxame: PSO binário/discreto com reparo.

CLONALG: mover obstáculos e garantir conectividade. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

25. Evolução de estratégias para jogo de cartas simplificado (quando apostar ou recuar).

Objetivo: maximizar retorno em simulações fixas.

Representação comum: vetor de limiares e ações da política.

Enxame: PSO misto com decodificador.

CLONALG: mudar limiares/regras de decisão. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

26. Otimização de layout de placa de circuito impresso (posição de componentes).

Objetivo: minimizar trilhas e interferências.

Representação comum: posições discretizadas de componentes.

Enxame: PSO discreto com reparo geométrico.

CLONALG: mover/trocar posição e eliminar sobreposição. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

27. Projeto de rede de distribuição de água/energia minimizando perdas e custos.

Objetivo: minimizar investimento/perdas respeitando demanda.

Representação comum: seleção de arestas e capacidades discretas.

Enxame: PSO binário/misto com reparo de conectividade.

CLONALG: ligar/desligar arestas e alterar capacidades. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

28. Otimização de layout de fábrica (posicionamento de máquinas).

Objetivo: minimizar deslocamento de material.

Representação comum: permutação de posições de máquinas.

Enxame: ACO para ordem/layout via decodificador.

CLONALG: trocar posições; reparar sobreposições. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

29. Design de antenas com segmentos discretos (liga/desliga).

Objetivo: maximizar indicador eletromagnético simulado.

Representação comum: máscara binária de segmentos ligados.

Enxame: PSO binário com função simulada de antena.

CLONALG: inverter segmentos; reparar limites. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

30. Projeto de portfólio de investimentos respeitando risco máximo e retorno esperado.

Objetivo: maximizar retorno ajustado ao risco.

Representação comum: vetor real de pesos com soma unitária.

Enxame: PSO contínuo com projeção para simplex.

CLONALG: redistribuir pesos e renormalizar. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

31. Configuração de parâmetros de um algoritmo de compressão para maximizar taxa.

Objetivo: minimizar tamanho + penalidade por tempo.

Representação comum: vetor misto de parâmetros de compressão.

Enxame: PSO misto com discretização.

CLONALG: alterar janela/bloco/nível. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

32. Alocação ótima de recursos em nuvem (containers em VMs).

Objetivo: minimizar uso, custo e violações de SLA.

Representação comum: identificador de VM para cada container.

Enxame: PSO discreto e reparo de capacidade.

CLONALG: migrar container entre VMs. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

33. Seleção de genes/variáveis em bioinformática para melhorar classificação.

Objetivo: maximizar F1-CV penalizando dimensionalidade.

Representação comum: máscara binária de genes/descritores.

Enxame: PSO binário.

CLONALG: inverter genes selecionados. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

34. Combinação de classificadores evoluindo pesos de ensemble.

Objetivo: minimizar log-loss ou erro de validação.

Representação comum: pesos reais não negativos de classificadores.

Enxame: PSO contínuo com normalização.

CLONALG: perturbar pesos e renormalizar. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

35. Otimização de protocolo de tratamento em simulação (sequência e doses).

Objetivo: minimizar custo simulado sujeito a limites.

Representação comum: sequência discretizada de intervenções em simulador.

Enxame: PSO misto/discreto com limites de simulação.

CLONALG: alterar intervenção ou dose simulada. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

36. Ajuste de parâmetros de modelos epidemiológicos (SIR/SEIR) a dados.

Objetivo: minimizar erro de ajuste a séries sintéticas/públicas.

Representação comum: vetor real de taxas do modelo SIR/SEIR.

Enxame: PSO contínuo em parâmetros positivos.

CLONALG: perturbar taxas dentro de limites. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

37. Evolução de fórmulas simples de QSAR (combinações de descritores).

Objetivo: minimizar erro de QSAR + complexidade.

Representação comum: coeficientes reais sobre descritores predefinidos.

Enxame: PSO contínuo com máscara opcional.

CLONALG: alterar coeficientes/termos. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

38. Clustering guiado por AG: representar atribuição de clusters e otimizar métricas.

Objetivo: minimizar dispersão intracluster.

Representação comum: centroides reais ou rótulos de clusters.

Enxame: PSO contínuo de centroides.

CLONALG: perturbar centroides ou reatribuir cluster. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

39. Otimização de layout de página web (posição de blocos de conteúdo).

Objetivo: minimizar violações de usabilidade predefinidas.

Representação comum: permutação/grade de blocos de interface.

Enxame: PSO discreto com decodificador de layout.

CLONALG: mover/trocar blocos; reparar sobreposição. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

40. Geração de escalas musicais ou progressões de acordes sob regras.

Objetivo: maximizar conformidade e diversidade.

Representação comum: sequência de acordes codificados por classes.

Enxame: ACO para transições harmônicas.

CLONALG: trocar acordes e reparar regras musicais. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

41. Personalização de recomendações evoluindo pesos para tipos de itens.

Objetivo: maximizar métrica de ranking em validação.

Representação comum: pesos reais de atributos de recomendação.

Enxame: PSO contínuo com normalização.

CLONALG: perturbar pesos de preferências. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

42. Otimização da sequência de perguntas em um formulário para reduzir abandono.

Objetivo: minimizar abandono simulado e extensão.

Representação comum: permutação de perguntas e condições de salto.

Enxame: ACO para ordem e caminho de perguntas.

CLONALG: trocar perguntas e alterar saltos. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

43. Ordenar conteúdos de um curso (sequência de aulas) maximizando aprendizado.

Objetivo: maximizar ganho simulado e minimizar violações.

Representação comum: permutação de conteúdos com pré-requisitos.

Enxame: ACO para sequência topológica.

CLONALG: mover tema preservando pré-requisitos. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

44. Montagem de prova a partir de banco de questões equilibrando dificuldade e tópicos.

Objetivo: minimizar desvio de dificuldade/tópicos.

Representação comum: máscara de questões e ordem em prova.

Enxame: PSO binário com reparo de cotas.

CLONALG: substituir questão mantendo cobertura. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

45. Distribuição de alunos em grupos de trabalho equilibrando rendimento e diversidade.

Objetivo: minimizar desigualdade e violação de regras.

Representação comum: atribuição de estudante por grupo.

Enxame: PSO discreto com reparo de tamanhos.

CLONALG: trocar alunos entre grupos. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

46. Otimização de rota de visita guiada em campus ou museu.

Objetivo: minimizar tempo/distância e violações.

Representação comum: permutação de pontos com janela de visita.

Enxame: ACO para ordenação dos pontos.

CLONALG: trocar paradas; reparar horários. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

47. Design de horários de monitoria para maximizar cobertura de alunos.

Objetivo: maximizar cobertura respeitando cargas.

Representação comum: atribuição de monitor por horário.

Enxame: PSO discreto com cobertura de demanda.

CLONALG: mover monitor entre faixas. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

48. Alocação de projetos de PFC para orientadores respeitando limites e temas.

Objetivo: maximizar aderência e respeitar capacidades.

Representação comum: atribuição projeto-orientador.

Enxame: PSO discreto com limites de orientações.

CLONALG: realocar/trocar projetos. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

49. Evolução de expressões matemáticas (symbolic regression) para ajustar dados.

Objetivo: minimizar erro + complexidade simbólica.

Representação comum: expressão de profundidade limitada codificada por gramática finita.

Enxame: PSO discreto sobre cromossomo de comprimento fixo que decodifica uma expressão por gramática finita (profundidade limitada).

CLONALG: substituir operador/operando/subárvore. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

50. Evolução de regras de um ecossistema artificial (predador–presa) para atingir equilíbrio.

Objetivo: minimizar distância de faixa de equilíbrio.

Representação comum: parâmetros de regras do simulador predador-presa.

Enxame: PSO contínuo/misto.

CLONALG: perturbar parâmetros/ativação de regras. AG: seleção, cruzamento e mutação compatíveis com essa codificação.

Apêndice B - Protocolo mínimo de implementação

1. **Definir instância:** entradas, tamanho máximo, gerador ou dados, restrições, semente e versão.
2. **Definir interface:** `decode(candidate)`; `evaluate(solution)` produz objetivo e violações; `repair(solution)` quando necessário.
3. **Definir três otimizadores:** AG com cruzamento específico; ACO ou PSO com adaptação demonstrada; CLONALG com clonagem e hipermutação específica.
4. **Testar casos pequenos:** solução conhecida, solução inválida, solução reparada, reexecução com semente registrada e checagem de restrições.
5. **Comparar:** usar orçamento comum de avaliações; no mínimo 10 execuções independentes por método; registrar valor objetivo bruto, percentual de viabilidade, custo computacional e convergência.

6. **Documentar:** escrever quais escolhas deste apêndice foram adotadas ou modificadas e justificar tecnicamente alterações. Nenhuma adaptação pode descaracterizar o problema numerado.